Project 3: Active Learning for Learning-to-Defer

Group

- Diya Raju Sonavale (674186)

Introduction

This project implements a human-AI collaboration system for topic classification on the AG News dataset (four classes: World, Sports, Business, Sci/Tech), exploring learning-to-defer and active learning. A classifier is trained to either answer independently or defer to a (simulated) human expert and the project investigates: (1) how good the classifier is alone, (2) how to simulate a realistically imperfect expert, (3) how to learn when deferring is worthwhile given full supervision and (4) how to learn the same thing under a constrained querying budget.

Task 1: Baseline Classifier

Method. Articles were represented using TF-IDF vectorization over unigrams and bigrams (50,000 features, English stop-words removed, sublinear term-frequency scaling), feeding a multinomial Logistic Regression classifier. Hyperparameters were selected via a validation sweep on a 90/10 split of the training data (test set held out entirely from tuning):

C	Validation accuracy
0.1	0.9096
0.3	0.9164
1	**0.9233**
3	**0.9237**
10	0.9194
30	0.9122

n-gram range	Validation accuracy
unigrams only	0.9178
unigrams + bigrams	0.9237
unigrams + bigrams + trigrams	0.9228

Justification. TF-IDF + Logistic Regression was chosen over heavier alternatives (embeddings, fine tuned transformers) because it trains in seconds on CPU, requires no additional heavy dependencies and reaches accuracy competitive with published shallow model baselines on this dataset, appropriate given the project's later stages (Tasks 3–4) require repeated retraining, where a slow base classifier would be impractical. C=3 and bigrams were chosen from the sweep above, trigrams were tested but rejected as they added noise rather than signal.

Results. Trained on the full 120,000 article training set, evaluated once on the official 7,600 article test set:

- **Test accuracy: 92.45%**
- Per-class F1: World 0.92, Sports 0.97, Business 0.90, Sci/Tech 0.90

Sports is easiest to classify (distinctive vocabulary). Business and Sci/Tech are hardest and most confused with each other, plausibly because both frequently discuss companies, products and financial figures.

Task 2: Simulated Expert

Method. The expert is a class-conditional noisy labeler, for an article of true class *c*, it returns the correct label with probability `EXPERT_ACCURACY[c]` and otherwise returns one of the other three classes chosen uniformly at random. Accuracy was deliberately set to be complementary to the Task 1 classifier's own weaknesses, 90% on Business and Sci/Tech (where the classifier struggles), 55% on World and Sports (where the classifier excels).

Justification. This is a standard technique from the noisy labels literature (class-conditional label noise). The complementary design (rather than uniform noise) is a deliberate simulation choice, not a claim about real human behavior. It is what makes the "should I defer?" decision genuinely non-trivial for later tasks. A uniformly worse or uniformly better expert would make deferral policy trivial ("never defer" or "always defer").

Results. On the test set:

- **Overall expert accuracy: 72.38%** (matching the expected average of 55/55/90/90 = 72.5%)
- Per-class recall closely tracked the design targets: World 54%, Sports 55%, Business 90%, Sci/Tech 91% - a direct sanity check that the simulation behaves as designed.
- F1: World/Sports 0.62 each (weak); Business/Sci-Tech 0.81 each (strong) — the intended inverse of the classifier's own profile.

Task 3: Learning to Defer

Method. With expert labels available for the full training set, a second binary classifier (the "rejector") was trained to predict, from article text alone, whether deferring to the expert would have been the better choice. Because the Task 1 classifier scores noticeably higher on its own training data (96.47%) than on genuinely new data (92.45% test accuracy). A normal ~4-point generalization gap training the rejector directly on the classifier's in-sample correctness would bias it toward underestimating how often deferral is needed. To avoid this, a throwaway "proxy" classifier (identical architecture) was trained on 80% of the training data and its honest, out of sample predictions on the remaining 20% (24,000 articles) were used to construct the rejector's training labels.

For each of these 24,000 articles, a "should defer" label was constructed by comparing the proxy classifier's correctness against the simulated expert's correctness on that article, label = 1 only if the expert was right *and* the classifier was wrong, label = 0 otherwise (including cases where both were right or both were wrong which ties default to "don't defer," on the reasoning that querying an expert is not free and should only happen when it demonstrably helps). The rejector itself (a Logistic Regression with balanced class weights) was then trained on these 24,000 articles' TF-IDF features (using the real, fully-trained Task 1 vectorizer, since this is what is actually deployed) against these labels.

At deployment: for each test article, the real Task 1 classifier produces its prediction and the rejector (text only, no ground truth) decides whether to defer. If so, the final answer is the simulated expert's answer.

Justification. The proxy classifier trick directly addresses a measured methodological risk (the 4-point in sample/out of sample gap) rather than an assumed one. The tie breaking default of "don't defer" reflects a real world assumption that expert queries carry some cost.

Results.

System	Accuracy
Classifier alone	92.45%
Expert alone	72.38%
Random deferral (same 12.2% rate)	90.04%
Team (learned deferral)	**93.22%**
Oracle ceiling (best possible)	97.97%

- The learned deferral policy deferred on 12.2% of test articles (929 of 7,600) and **outperformed the classifier alone** (+0.77 points) and **clearly outperformed random deferral at the identical rate** (93.22% vs. 90.04%) — confirming the rejector learned a genuinely informative signal, not just beneficial noise-mixing (random deferral actually *hurts* accuracy here, since the expert is weaker on average).
- Quality of deferral check: among the 929 deferred articles, the classifier would have been correct only 77.7% of the time (down from its overall 92.45%), while the expert was correct 84.1% of the time (up from its overall 72.38%) confirming the rejector is correctly steering toward classifier weak/expert strong territory, if imperfectly.
- The team captured only about 14% of the theoretically achievable headroom between the classifier alone and the oracle ceiling ((93.22−92.45)/(97.97−92.45)), an honest limitation, since the rejector, unlike the oracle, only has access to article text, not the true label.

Task 4: Active Learning for Expert Competence Discovery

Method. This task removes the assumption of full expert-label access, only a limited number of expert queries are permitted, and the goal is to choose them wisely. To stay comparable with Task 3, the same 24,000 article held-out pool was reused. Two query strategies were compared, both proceeding in 8 rounds of 500 queries (4,000 total, one-sixth of Task 3's full pool):

- **Active strategy**: the first round queries the classifier's least-confident articles; every subsequent round queries whichever remaining article the *current* rejector is most unsure whether to defer on (i.e., its predicted "should defer" probability is closest to 0.5). This directly targets reducing uncertainty about the deferral decision itself, rather than general classification difficulty.
- **Random strategy** (baseline): identical budget schedule, but each round's queries are chosen uniformly at random.

Justification. Using the rejector's own uncertainty (rather than a fixed criterion) after the first round is what makes this a genuinely *active* (adaptive) strategy, it targets the assignment's stated goal of learning "the

expert's competence profile" specifically, rather than generic classification difficulty, which does not change round to round.

Results — a negative but well-diagnosed finding.

First run (uncalibrated, default 0.5 decision threshold):

Round (n queried)	Active team accuracy	Random team accuracy
500	89.25%	92.43%
1000	92.38%	92.47%
4000	92.57%	92.62%

At round 1, the active strategy catastrophically underperformed, worse than the classifier alone. Diagnosis: the round-1 batch (the 500 most classifier-uncertain articles) had a "should defer" rate of 39.2%, versus the true population rate of ~6.3% (confirmed by inspecting random's batches, which stayed close to 6% throughout). This taught the rejector that deferral is far more common than it really is, causing it to over-defer on 19% of the *entire* test set including many articles the classifier would have answered correctly. This is a textbook active-learning sampling bias failure, deliberately querying "hard" examples produces training data that does not represent the deployment distribution.

Second run (calibrated): the decision threshold was adjusted so the predicted deferral rate on each round's training batch matched the known population rate (6.254%, from Task 3), instead of the default 0.5 cutoff.

Round (n queried)	Active team accuracy	Active deferral rate	Random team accuracy
500	92.45%	0.00%	92.51%
4000	92.45%	0.07%	92.50%

This fixed the over-deferral catastrophe (active no longer crashes below baseline) but revealed a second, deeper issue: the active strategy now essentially never defers (deferral rate ≈0%), so its team accuracy is just the classifier's own accuracy, flat across all rounds. The calibration threshold, computed from the batch's own (systematically skewed) probability distribution, did not transfer to the differently distributed test set which is a subtler instance of the same underlying problem: **uncertainty-based querying produces non-representative training data, which undermines not just raw training but also naive attempts to calibrate against it.**

Conclusion. Across both runs, the active strategy never clearly outperformed random querying. This is reported as a genuine, defensible finding rather than a failed exercise. It illustrates a real tension in active learning, a strategy well-suited to flagging *individual* hard cases is not automatically well-suited to estimating *aggregate* statistics (like how often to defer) from the resulting sample, because that sample by construction is not representative of the population it will be deployed on.

Task 5 (Optional): Interactive Human-in-the-Loop Interface

Method. A live web page lets a real user play the expert role. To keep the interface responsive, a pool of 300 articles (the same signal pool cases the classifier finds most difficult) and the transformed test set features were precomputed offline. On each visit, the user is shown one article at a time. The first one is

chosen by classifier uncertainty, subsequent ones chosen by the (session-specific, continuously retrained) rejector's uncertainty, mirroring the Task 4 active strategy exactly. After each answer, the page shows immediate feedback (the user's answer, the classifier's answer and the true topic) and the resulting live team accuracy, evaluated on the full official test set.

Results. Verified with simulated human behavior (a scripted "always correct" run and a realistic "80% accurate" run, 20 rounds each), no errors across either run and the same early round instability observed in Task 4 (a rejector trained on very few examples produces volatile, sometimes-poor decisions) reproduced live, exactly as expected, confirming the interactive version behaves consistently with the offline experiments rather than diverging from them.

Overall Discussion

The project demonstrates a full pipeline from independent classification, through simulated expertise, to collaborative decision-making. The clearest positive result is Task 3. Given full supervision, a simple learned rejector genuinely improves on both the classifier and the expert alone and clearly outperforms naive random deferral, showing it captures real, non-trivial signal. The clearest negative (but equally valuable) result is Task 4. Under a constrained querying budget, naive uncertainty-based active learning failed to beat random querying, for a well-understood statistical reason (sampling bias) that persisted even after a targeted calibration fix. Together, these results suggest that while learning *when* to defer is tractable with sufficient supervision, *efficiently discovering* that policy under a limited labeling budget is considerably harder than a standard active-learning intuition would suggest.